SIMATS ENGINEERING

ASSESSMENT TOOL 2 — ANSWERS

Course: Artificial Intelligence **Course Code:** CSA17

Topic: Constraint-Based Problem Solving

Name: _____ **Reg. No.:** _____

Note: Question 2 does not include a specific grid drawing in the supplied question paper. Therefore, the grid below is an illustrative example used to demonstrate the method. The actual path may differ if your instructor has provided a separate grid.

1. Hospital Doctor–Shift Assignment Using Backtracking Search

Variables: D1, D2, D3. **Domain:** Morning (M), Afternoon (A), Night (N).

Constraints: (1) D1 ≠ N; (2) shift(D2) must be earlier than shift(D3); (3) one doctor per shift; (4) D3 ≠ M; (5) every doctor receives exactly one shift.

Backtracking procedure: Assign one variable at a time. After each assignment, check all constraints that can already be evaluated. If a constraint is violated, undo that assignment and try the next value.

Intermediate steps (variable order D1 → D2 → D3):

Step	Trial / assignment	Constraint check	Decision
1	D1 = Morning	D1 is not Night — valid	Keep
2	D2 = Afternoon	D2 is after Morning; no conflict so far	Keep
3	D3 = Night	D2 (Afternoon) is earlier than D3 (Night); D3 is not Morning; all shifts are unique	Accept

Final valid schedule: D1 — Morning; D2 — Afternoon; D3 — Night.

Why it is valid: D1 is not assigned to Night. D2 works before D3 (Afternoon is before Night). No two doctors share a shift, D3 is not in Morning, and each doctor has exactly one shift.

Backtracking note: If a trial violates any constraint, the search returns to the previous variable, changes its value, and continues. For example, D1 = Night would be rejected immediately because it violates constraint (i).

2. Robot in a 5 × 5 Grid: BFS and Manhattan Distance

Because the paper does not specify the locations of S, G, or obstacles, use this example grid. Movement is allowed only Up, Down, Left, and Right; each move costs 1. '#' denotes an obstacle.

S	.	#	.	.
.	.	#	.	#
#	.	.	.	.
.	#	#	.	.
.	.	.	.	**G**

Coordinates: Rows and columns are numbered 1–5 from the top-left. Start S = (1,1); Goal G = (5,5).

BFS method: Use a FIFO queue. Begin with S, expand its legal unvisited neighbours in the fixed order Right, Down, Left, Up, and record each node's parent. Ignore obstacles and already visited cells. Continue until G is discovered; reconstruct the route by following parent links backwards.

Illustrative BFS queue/expansion trace:

Expansion	Node removed from queue	New legal nodes added (Right, then Down, Left, Up)
1	(1,1)	(1,2), (2,1)
2	(1,2)	(2,2) [right is blocked; left already visited]
3	(2,1)	No new node [down already visited; left is blocked]
4	(2,2)	(3,2) [right is blocked; up already visited]
5	(3,2)	(3,3) [down is blocked; left is blocked]
6	(3,3)	(3,4)
7	(3,4)	(4,4), (2,4), (3,5)
8	(4,4)	(5,4), (4,5)
9	(2,4)	(1,4)
10	(3,5)	(4,5) may already be discovered; (2,5) is blocked
11	(5,4)	(5,5) = G; goal found

One optimal path: (1,1) → (1,2) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4) → (5,4) → (5,5).

Total path cost: 8 moves × 1 = **8**.

Manhattan distance: h(n) = |row(n) – row(G)| + |column(n) – column(G)|. At S, h(S) = |1–5| + |1–5| = 8. This heuristic estimates distance while ignoring obstacles.

Important algorithm distinction: Standard BFS expands nodes by queue order and does not use a heuristic to prioritize nodes. With equal move costs, BFS finds a shortest path. If the heuristic must guide which node is expanded next, A* search (using f(n) = g(n) + h(n)) is the appropriate alternative.

3. Autonomous Rescue Robot in a Partially Observable Building

(a) Analysis of constraints and their effect

• **Four-direction movement:** The robot can move only up, down, left, or right, so diagonal shortcuts are not allowed.

• **Obstacles:** Blocked cells are forbidden states. The robot must sense and avoid them.

• **Limited sensing:** It can observe only adjacent cells, so it cannot know the complete map initially. It must make a local decision and update its map after each observation.

• **Movement cost:** Each move costs 1.

• **Risky zones:** Entering a risky cell adds 2 to the normal movement cost. Therefore, entering such a cell costs 1 + 2 = 3.

• **Cost and safety objective:** The robot should avoid blocked cells and select a safe route with minimum accumulated cost, rather than simply choosing the route with the fewest steps.

• **Online updating:** The robot revises its known map when new information is sensed; its plan may change if a newly observed obstacle or risk changes the best route.

(b) Proposed approach: Online Search Agent with Uniform Cost Search (UCS)

UCS is suitable for choosing a least-cost route because all step costs are non-negative but are not equal when risky zones are present. The robot can use UCS over its currently known map and re-plan as new cells are observed.

1. Initialize the robot's current position as S. Mark only the cells it has observed; unknown cells remain unknown rather than being assumed safe.

2. Sense the four adjacent cells. Record whether each is open, blocked, or risky.

3. Add reachable known cells to a priority queue ordered by accumulated path cost g(n). A normal move adds 1; a move into a risky zone adds 3.

4. Remove the lowest-cost state from the queue and consider its legal neighbours. Update a neighbour's best known cost and parent if a cheaper route is found.

5. If the goal is reached, reconstruct the least-cost route using parent links and follow it.

6. After each movement, sense adjacent cells and update the map. If the new information invalidates or increases the cost of the planned route, run UCS again from the current position.

7. Stop when the survivor at G is reached, and report the route and its total cost.

Cost calculation: For a route with N normal moves and R entries into risky zones, total cost = N + 2R, provided N counts every movement including the risky entries. Equivalently, add 1 for each move and an additional 2 for every risky-zone entry.

(c) AI concepts demonstrated

Intelligent agent: The robot perceives the environment using adjacent-cell sensors, maintains an internal map, chooses actions using a search algorithm, and acts by moving toward the survivor.

Rationality: A rational robot selects the action expected to achieve the goal with the lowest known path cost while avoiding blocked cells and accounting for risk. Its decisions are based on the information available at that time; limited sensing means it cannot guarantee knowledge of unseen cells.

Environment type:

• Partially observable: only adjacent cells can be sensed.

• Sequential: each move affects the robot's future position and choices.

• Dynamic from the agent's perspective: its internal knowledge changes as it explores, even if the building itself does not change.

• Discrete: rooms/cells and actions are represented as distinct states and moves.

• Single-agent: the robot is the decision-making agent in this problem.

• Stochasticity is not specified; if movements and sensing are assumed reliable, the model can be treated as deterministic.

(d) Justification for the chosen approach

Uniform Cost Search is appropriate because the objective is to minimize total cost, and risky-zone entries have a higher cost than ordinary moves. Unlike ordinary BFS, UCS prioritizes the lowest accumulated cost and can therefore prefer a longer route if that route avoids expensive risky cells. Combining UCS with online sensing allows the robot to update its map and re-plan when it discovers new obstacles or risky zones. The approach respects movement restrictions, avoids known blocked cells, and provides a systematic least-cost solution based on the information currently available.

Conclusion

Backtracking finds a doctor assignment by testing and rejecting inconsistent choices. BFS finds a shortest path when every move has equal cost, while Manhattan distance is a heuristic estimate rather than a BFS priority rule. For the rescue robot, online sensing combined with Uniform Cost Search is better suited to minimizing a path cost that increases in risky zones.